PostgreSQL — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Installation

02 Databases, Schemas and psql Commands

03 Data Types and Creating Tables

04 CRUD and RETURNING

05 Filtering, Sorting, Pagination

06 Constraints

Intermediate (6 topics)

07 Joins and Set Operations

08 Aggregates, GROUP BY and FILTER

09 Subqueries and CTEs (Recursive)

10 Window Functions

11 JSONB and Arrays

12 Views and Materialized Views

Advanced (6 topics)

13 Indexes: B-tree, GIN, GiST, BRIN

14 Transactions, MVCC and Locking

15 Functions, Procedures and Triggers (PL/pgSQL)

16 Full-text Search

17 Roles, Permissions and Row-Level Security

18 Extensions

Master (4 topics)

19	Performance Tuning and VACUUM
20	Backup, Restore and PITR
21	Replication, HA and Partitioning
22	Real Project: Multi-tenant SaaS Schema

Beginner Level

01. Introduction and Installation

PostgreSQL is a powerful open-source object-relational database known for standards compliance, extensibility and reliability. Use `psql` or pgAdmin.

```
psql -U postgres
\l
```

02. Databases, Schemas and psql Commands

Create databases and schemas; useful meta-commands: `\dt`, `\d table`, `\c db`, `\q`.

```
CREATE DATABASE shop;
\c shop
CREATE SCHEMA sales;
```

03. Data Types and Creating Tables

`integer`, `bigserial`, `numeric`, `text`, `varchar`, `boolean`, `date`, `timestamptz`, `uuid`, `jsonb`, arrays, enums.

```
CREATE TABLE customers (
  id bigserial PRIMARY KEY,
  name text NOT NULL,
  created_at timestamptz DEFAULT now()
);
```

04. CRUD and RETURNING

Standard CRUD with the handy `RETURNING` clause.

```
INSERT INTO customers (name) VALUES ('Priya') RETURNING id;
```

05. Filtering, Sorting, Pagination

`WHERE`, `ILIKE`, `ORDER BY`, `LIMIT/OFFSET`, and keyset pagination for large tables.

```
SELECT * FROM customers WHERE name ILIKE 'p%' ORDER BY id LIMIT 20;
```

06. Constraints

Primary, foreign, unique, check and exclusion constraints keep data valid.

```
ALTER TABLE orders ADD CONSTRAINT qty_pos CHECK (qty > 0);
```

Intermediate Level

07. Joins and Set Operations

All joins plus `UNION`, `INTERSECT`, `EXCEPT`; `LATERAL` joins.

```
SELECT c.name, o.total FROM customers c LEFT JOIN orders o ON o.customer_id = c.id;
```

08. Aggregates, GROUP BY and FILTER

Aggregates with `FILTER`, `GROUPING SETS`, `string_agg`, `array_agg`.

```
SELECT count(*) FILTER (WHERE total > 1000) FROM orders;
```

09. Subqueries and CTEs (Recursive)

CTEs simplify queries; recursive CTEs traverse trees such as categories.

```
WITH RECURSIVE t AS (
  SELECT id, parent_id FROM categories WHERE parent_id IS NULL
  UNION ALL
  SELECT c.id, c.parent_id FROM categories c JOIN t ON c.parent_id = t.id
) SELECT * FROM t;
```

10. Window Functions

`ROW_NUMBER`, `RANK`, `LAG/LEAD`, running totals.

```
SELECT id, total, sum(total) OVER (ORDER BY id) FROM orders;
```

11. JSONB and Arrays

Store and query semi-structured data with `jsonb` operators and GIN indexes.

```
SELECT data->>'city' FROM profiles WHERE data @> '{"vip": true}';
```

12. Views and Materialized Views

Views store queries; materialized views store results and can be refreshed.

```
CREATE MATERIALIZED VIEW monthly AS SELECT date_trunc('month', created_at) m, sum(total) FROM
orders GROUP BY 1;
```

Advanced Level

13. Indexes: B-tree, GIN, GiST, BRIN

Choose index types per workload; partial and covering indexes; verify with `EXPLAIN (ANALYZE, BUFFERS)`.

```
CREATE INDEX CONCURRENTLY idx_orders_cust ON orders (customer_id);
```

14. Transactions, MVCC and Locking

MVCC lets readers not block writers; isolation levels; row locks (`FOR UPDATE`), advisory locks, deadlocks.

```
BEGIN;
SELECT * FROM acc WHERE id = 1 FOR UPDATE;
COMMIT;
```

15. Functions, Procedures and Triggers (PL/pgSQL)

Server-side logic with PL/pgSQL, triggers and event triggers.

```
CREATE FUNCTION touch() RETURNS trigger AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END $$ LANGUAGE plpgsql;
```

16. Full-text Search

`tsvector`, `tsquery`, GIN indexes for fast search.

```
SELECT * FROM posts WHERE to_tsvector('english', body) @@ to_tsquery('postgres & index');
```

17. Roles, Permissions and Row-Level Security

Roles and grants; Row-Level Security policies for multi-tenant data.

```
ALTER TABLE docs ENABLE ROW LEVEL SECURITY;
```

18. Extensions

`pg_stat_statements`, `postgis`, `pgcrypto`, `pg_trgm`, `pgvector`.

```
CREATE EXTENSION pg_trgm;
```

Master Level

19. Performance Tuning and VACUUM

Autovacuum, `ANALYZE`, `work_mem`, `shared_buffers`, connection pooling (PgBouncer), query plans.

```
VACUUM (ANALYZE) orders;
```

20. Backup, Restore and PITR

`pg_dump`, `pg_restore`, base backups and WAL archiving for point-in-time recovery.

```
pg_dump -Fc shop > shop.dump
pg_restore -d shop shop.dump
```

21. Replication, HA and Partitioning

Streaming and logical replication, Patroni failover, declarative partitioning.

```
CREATE TABLE logs (...) PARTITION BY RANGE (created_at);
```

22. Real Project: Multi-tenant SaaS Schema

Schemas/RLS, indexes, migrations, reporting views and backups for a SaaS app.

Interview Question Bank

Q1. What is MVCC?
Multi-Version Concurrency Control keeps row versions so readers and writers do not block each other.

Q2. PostgreSQL vs MySQL?
PostgreSQL has richer SQL features, extensions and types (jsonb, arrays); MySQL is simpler and widely used.

Q3. What is VACUUM?
Removes dead row versions and updates statistics to reclaim space.

Q4. What index types exist?
B-tree, Hash, GIN, GiST, SP-GiST, BRIN.

Q5. `json` vs `jsonb`?
`jsonb` is binary, indexable and faster to query; `json` stores exact text.

Q6. What is a CTE?
A named temporary result set using `WITH`, possibly recursive.

Q7. What is a materialized view?
A view whose results are stored and refreshed on demand.

Q8. How do you analyse a query?
`EXPLAIN (ANALYZE, BUFFERS)` shows plan and real timings.

Q9. What is Row-Level Security?
Policies restricting which rows a role can see or change.

Q10. What is WAL?
Write-Ahead Log ensuring durability and enabling replication and recovery.

Capstone Project Ideas

1. Online shop schema
2. Analytics with window functions
3. Multi-tenant SaaS with RLS
4. Full-text blog search
5. Geo app with PostGIS